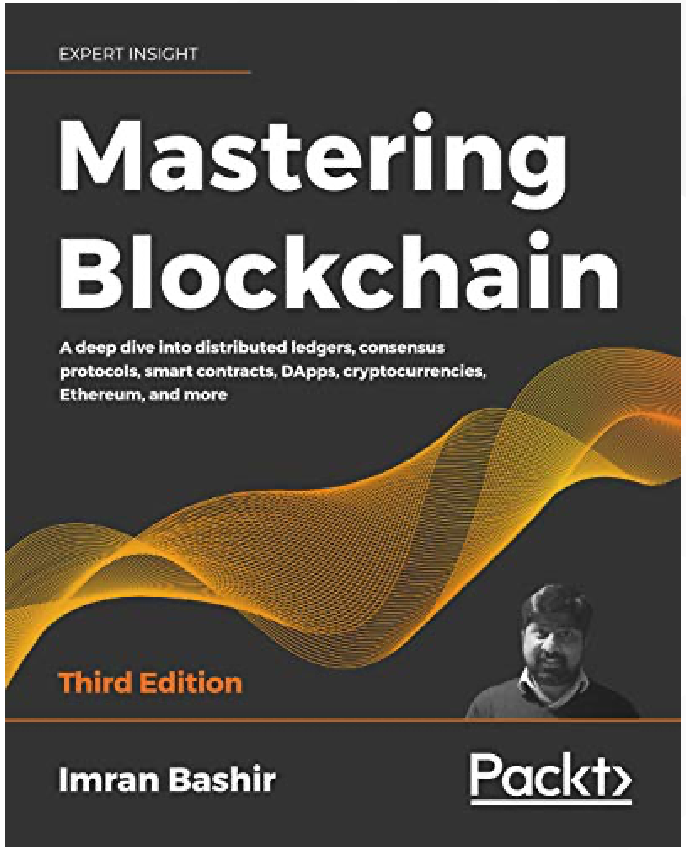
Mastering Blockchain

Third Edition

Chapter 4, Public Key Cryptography



Outline

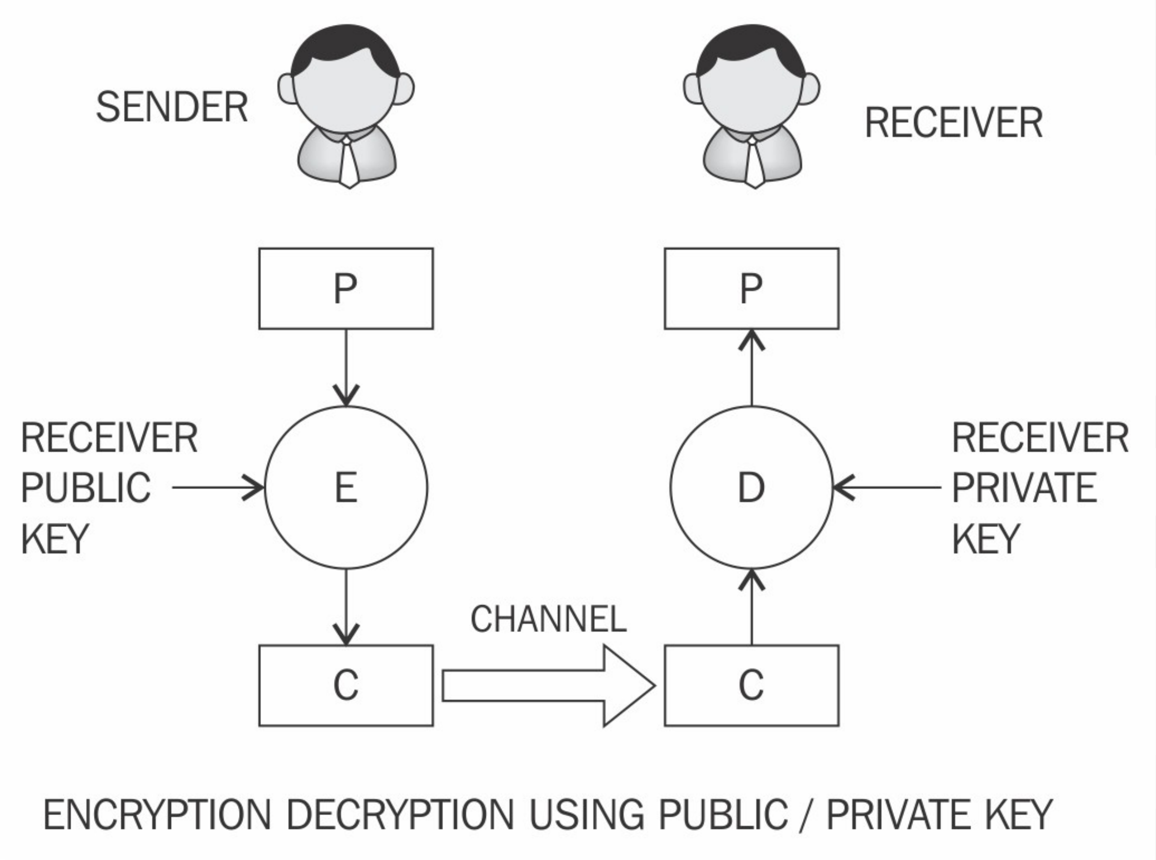
- Mathematics
- Asymmetric cryptography
- Cryptographic constructs and blockchain
- Zero-knowledge proofs
- Digital signatures
- Merkle trees, Patricia trees and distributed hash tables

Mathematics

- Modular arithmetic
- Sets
- Fields
- Finite fields
- Prime fields
- Groups
- Abelian groups
- Rings
- Cyclic groups
- Order

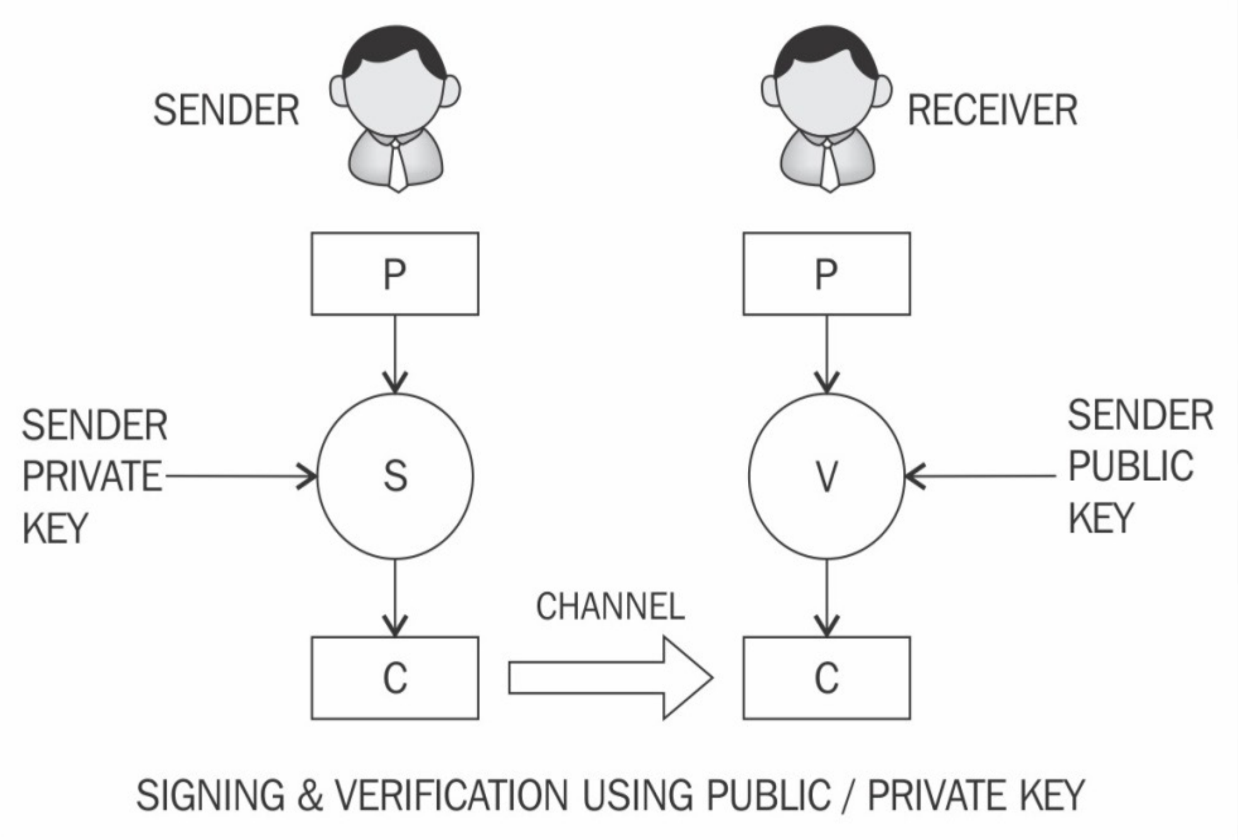
Asymmetric cryptography

Encryption and decryption: where the sender uses the receiver’s public key to encrypt, and the receiver uses their private key to decrypt the message.



Signing and verification

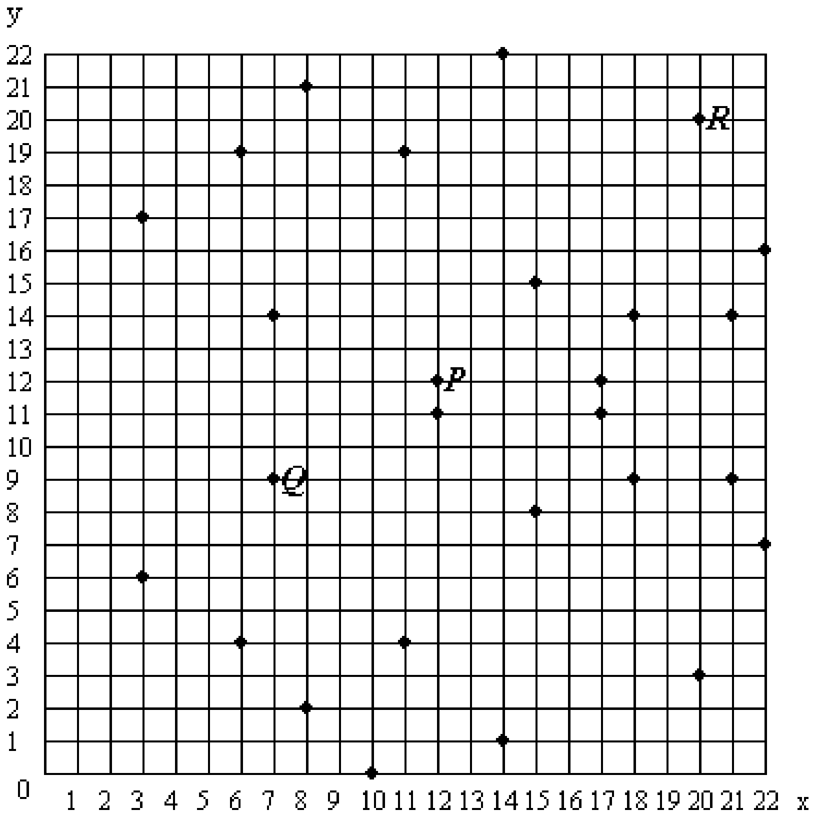
Signing and verification: where the sender uses their private key to sign, and the receiver uses the sender’s public key to verify that the message has indeed been sent from the sender.



RSA

- Developed in 1977 by Ron **R**ivest, Adi **S**hamir, and Leonard **A**delman.
- Key pairs are generated by performing the following steps:
    - Modulus generation
    - Generate the co-prime
    - Generate the public key
    - Generate the private key:

Elliptic Curve Cryptography – Point addition



$y^2 = x^3 + 7x + 11$ over F_{23}

27 solutions

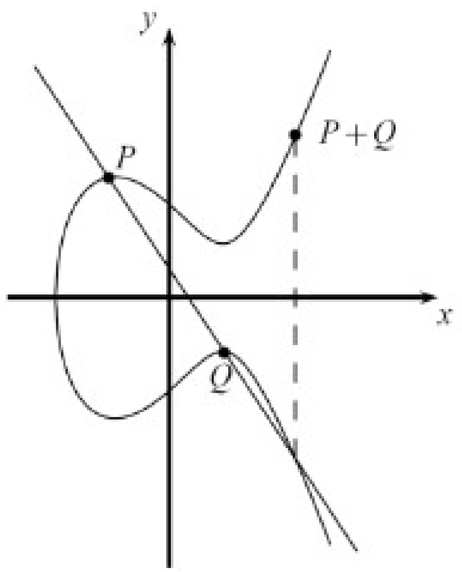
P (12, 12)

Q (7, 9)

R (20, 20)

$$
\begin{aligned}
l &= (y_P - y_Q) * (x_P - x_Q)^{-1} \bmod p \\
&= 3 * 5^{-1} \bmod 23 \\
&= 3 * 14 \bmod 23 \\
&= 19 \\
x_R &= l^2 - x_P - x_Q \bmod p \\
&= 361 - 12 - 7 \bmod 23 \\
&= 20 \\
y_R &= -y_P + l * (x_P - x_R) \bmod p \\
&= -12 + 19 * (12 - 20) \bmod 23 \\
&= 11 + 19 * 15 \bmod 23 \\
&= 11 + 9 \bmod 23 \\
&= 20
\end{aligned}
$$

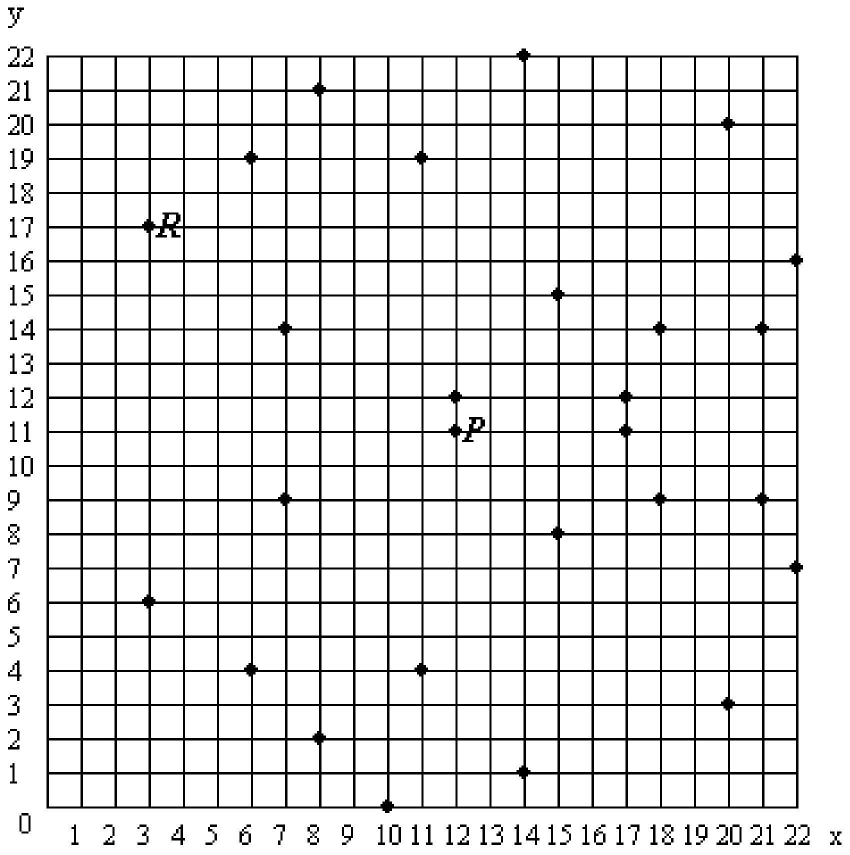
$P + Q = R = (20, 20)$.



Point addition over R

$$y^2 = x^3 + ax + b$$

Elliptic Curve Cryptography – Point doubling



$y^2 = x^3 + 7x + 11$ over F_{23}

27 solutions

$P\ (12, 11)$

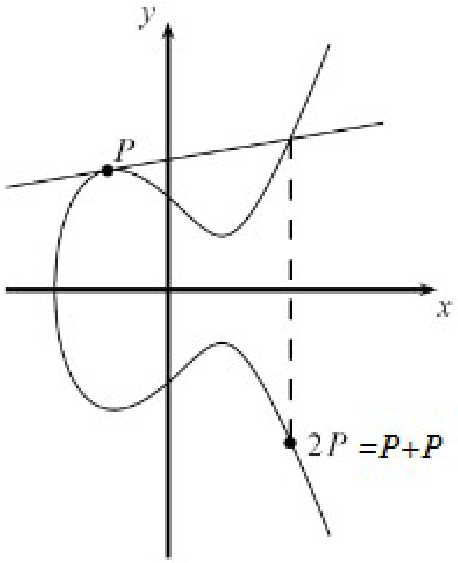
$R\ (3, 17)$

$$l = (3x_P^2 + a) * (2y_P)^{-1} \bmod p$$
$$= 439 * 22^{-1} \bmod 23$$
$$= 2 * 22 \bmod 23$$
$$= 21$$

$$x_R = l^2 - 2x_P \bmod p$$
$$= 441 - 24 \bmod 23$$
$$= 3$$

$$y_R = -y_P + l * (x_P - x_R) \bmod p$$
$$= -11 + 21 * (12 - 3) \bmod 23$$
$$= 12 + 21 * 9 \bmod 23$$
$$= 12 + 5 \bmod 23$$
$$= 17$$

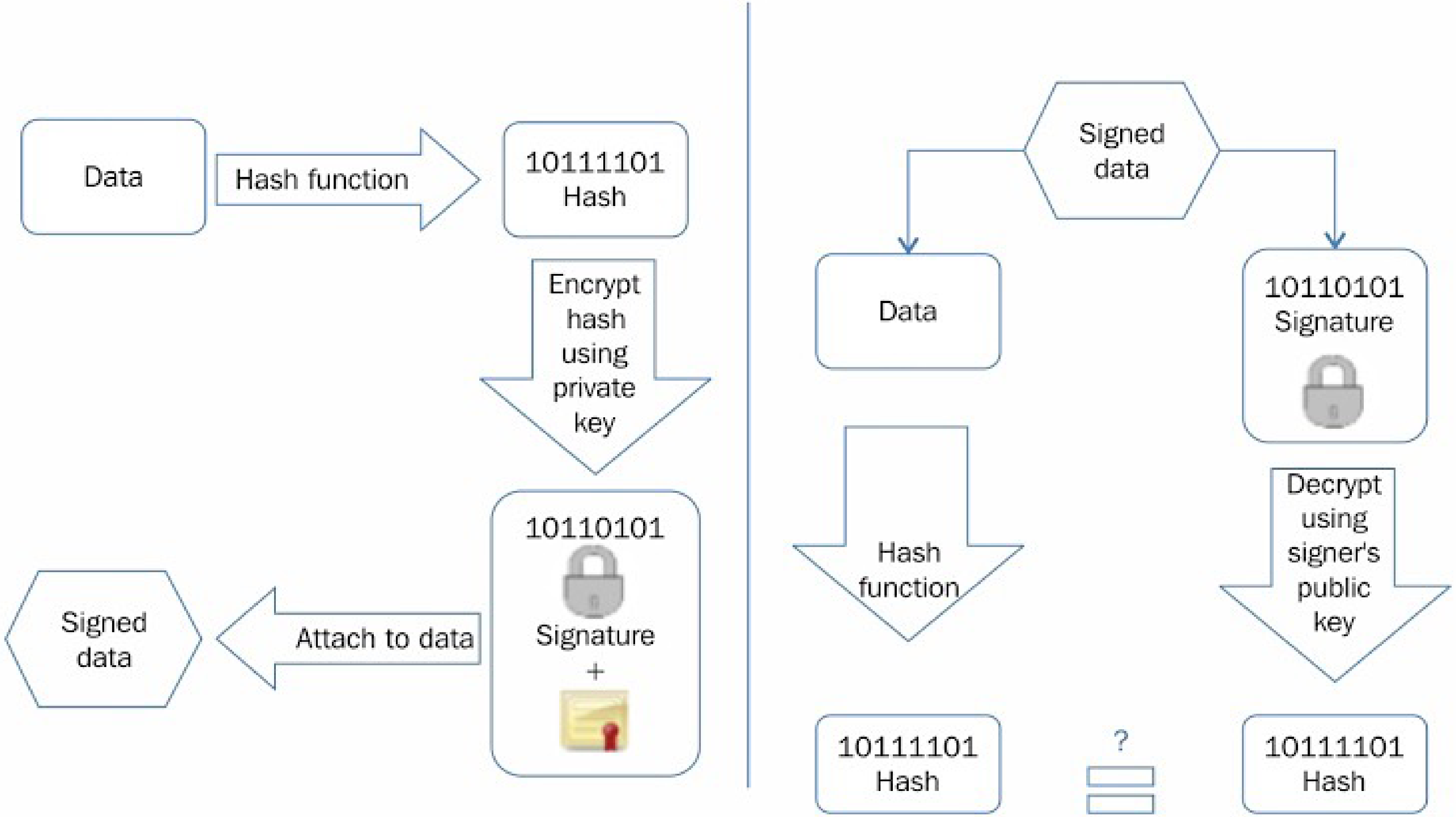
$2P = R = (3, 17)$.



point doubling over R

$$y^2 = x^3 + ax + b$$

Digital signatures



Digital signing (left) and verification process (right)

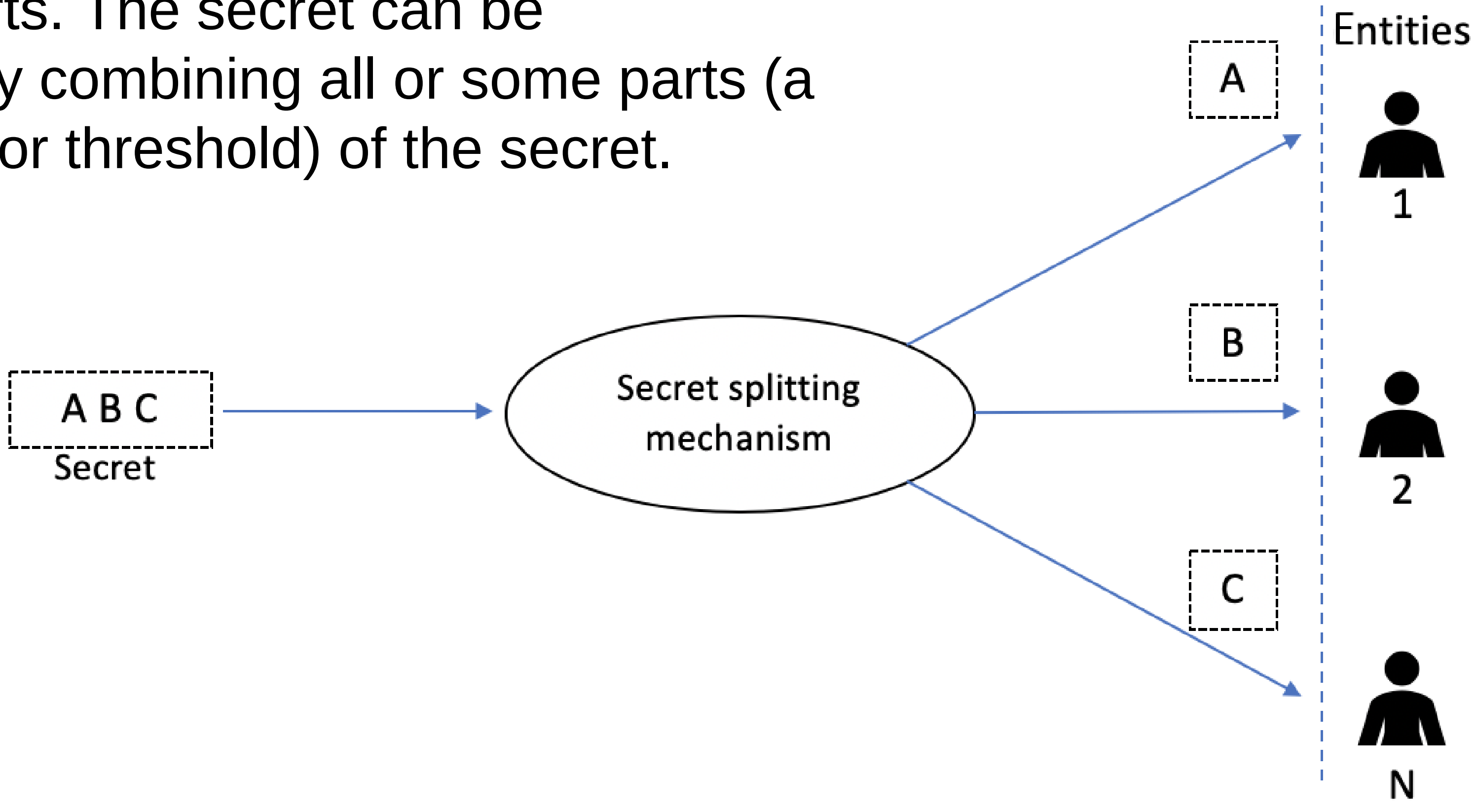
Cryptographic constructs and blockchain technology

Some advanced cryptography schemes that can be used in blockchain to enhance performance, security and privacy:

- Homomorphic encryption
- Signcryption

Secret sharing scheme

Secret sharing is the mechanism of distributing a secret among a set of entities. All entities within a set get a unique part of the secret after it is split into multiple parts. The secret can be reconstructed by combining all or some parts (a certain number or threshold) of the secret.



Zero-knowledge proofs

Three properties of zero-knowledge proofs are:

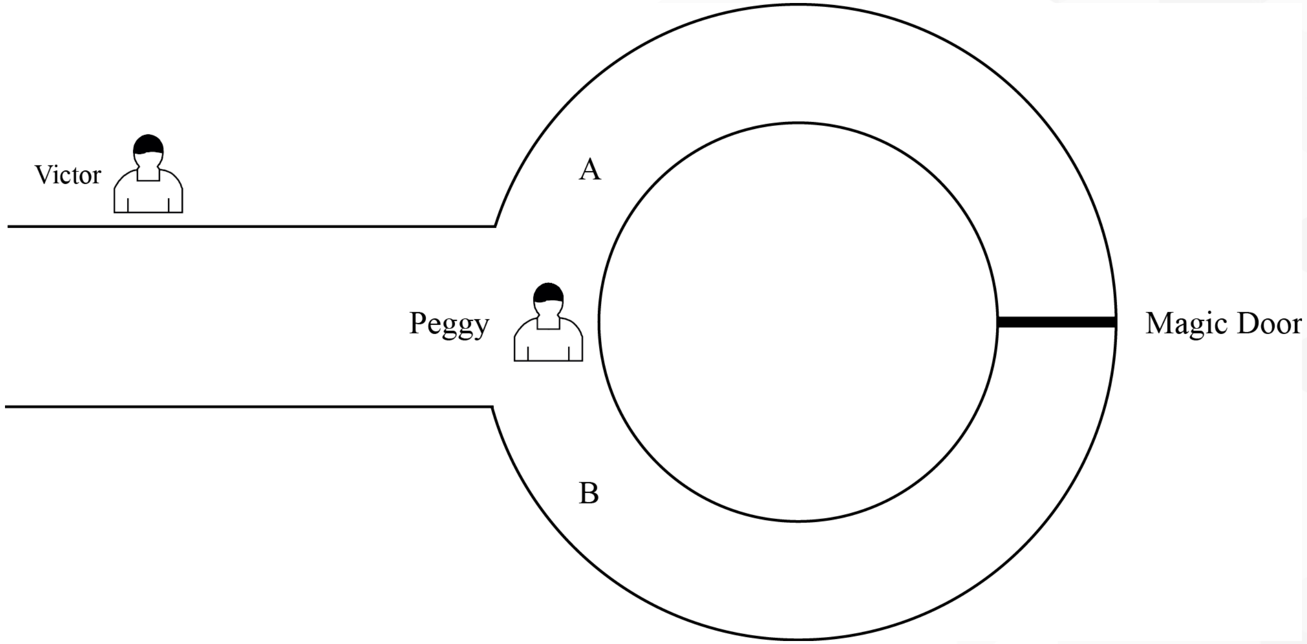
- Completeness
- Soundness
- Zero-knowledge property

Analogy of the zero-knowledge mechanism

Peggy knows a secret magic word to open the door in a cave, which is shaped like a ring. It has a split entrance and has a **magic door** in the middle of the ring that blocks the other side.

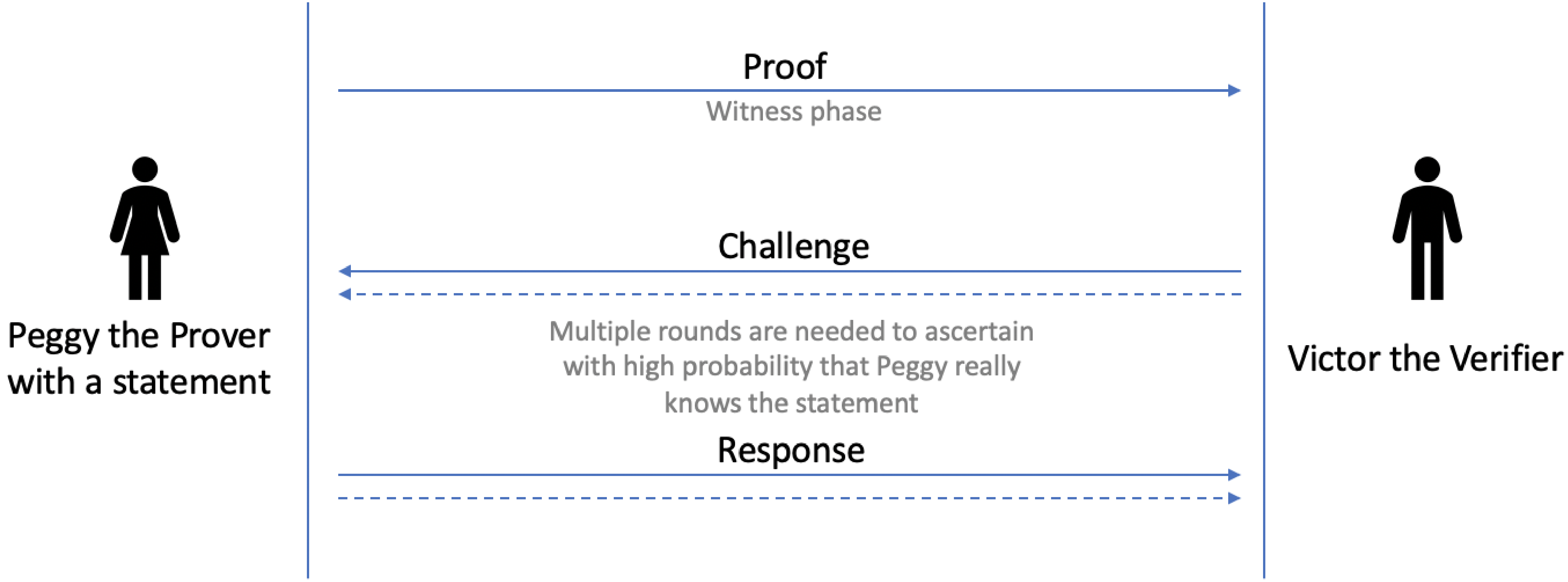
To find out whether Peggy knows the secret or not:

1. First, Victor waits outside the main cave entrance and Peggy goes in the cave.
2. Peggy randomly chooses either the **A** or **B** entrance to the cave.
3. Now, Victor enters the cave and shouts either **A** or **B** randomly, asking Peggy to come out of the exit he named.
4. Victor records which exit Peggy comes out from.



If Peggy repeatedly manages to emerge from the entrance that Victor has named, then it is highly probable that Peggy does know the secret to open the magic door.

ZKP scheme


Proof
Witness phase
Peggy the Prover with a statement
Challenge
Multiple rounds are needed to ascertain with high probability that Peggy really knows the statement
Response
Victor the Verifier

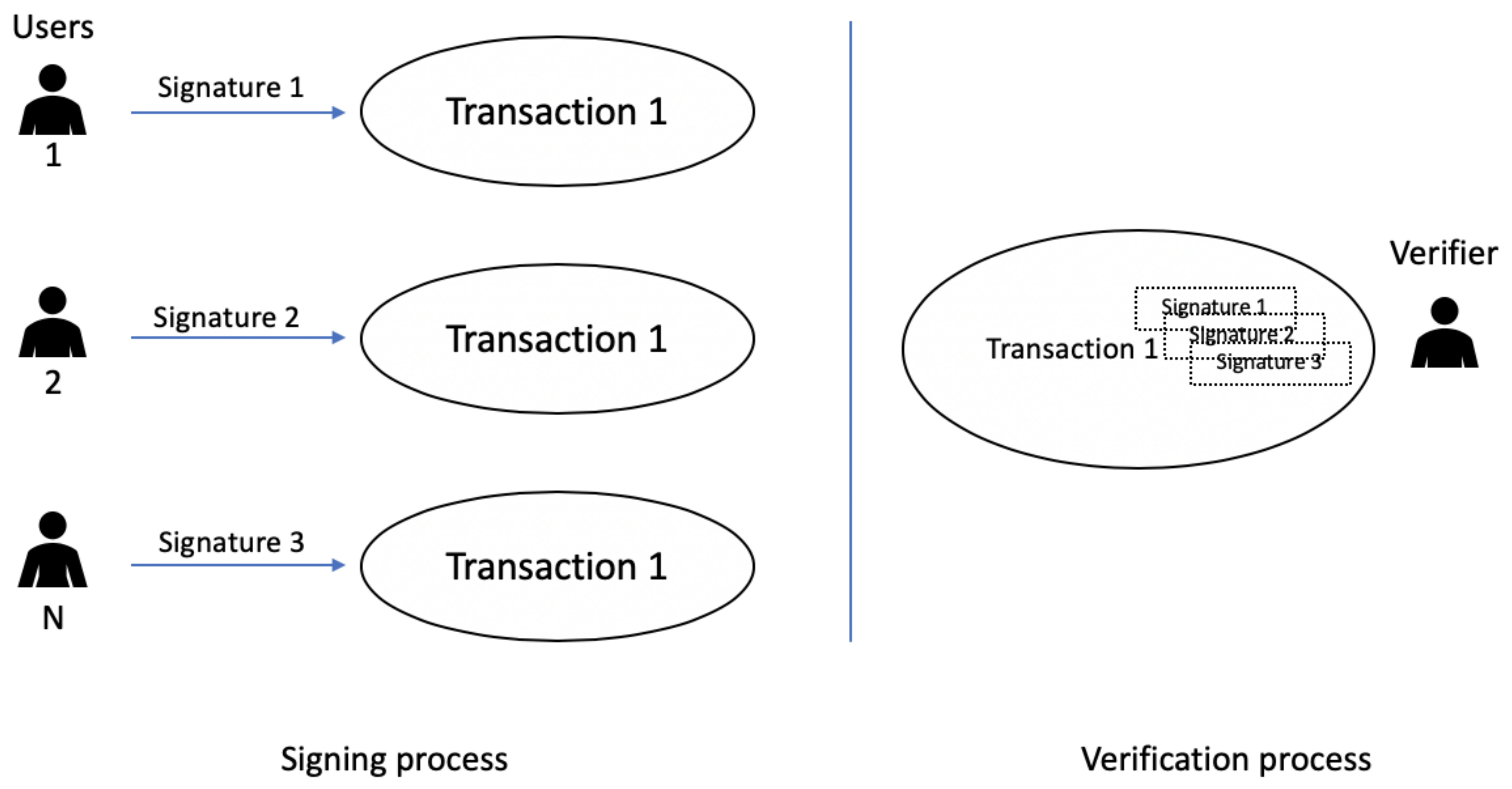
zk-SNARKs

Zk-SNARKs have several properties:

- **Zero knowledge (zk)**: This property allows a prover to convince a verifier that an assertion (statement) is true without revealing any information about it.
- **Succinct (S)**: This property allows for a small proof that is very quick to verify and is succinct.
- **Non-interactive (N)**: This property allows the prover to prove a statement without any interaction with the verifier.
- **ARguments of Knowledge (ARKs)**: These are the arguments to convince the verifier that the prover's assertion is true. In the case of ARKs, the prover has a computational soundness property, meaning that the prover is computationally bounded, and it is computationally infeasible for it to cheat the verifier.

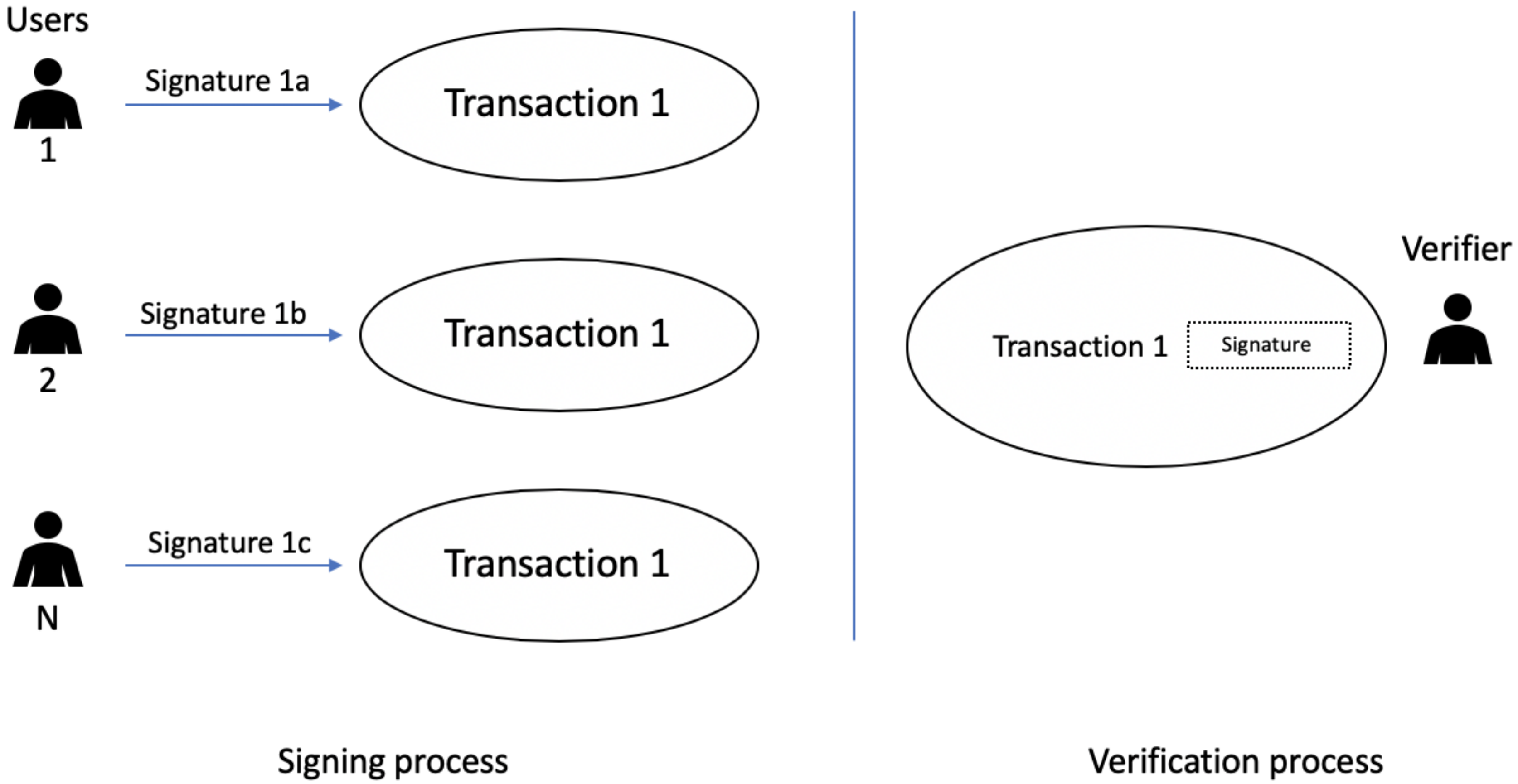
Multi-Signatures

The signing process is on the left-hand side, where N is the number of different users, who, holding N unique signatures, signs a single transaction. When the verifier receives it, all the signatures in it are individually verified.



Threshold signatures

The signing process is on the left-hand side, where N is the number of different users, who, holding different parts (shares) of a digital signature, sign a single transaction. When the validator or verifier receives it, only one signature needs to be verified.

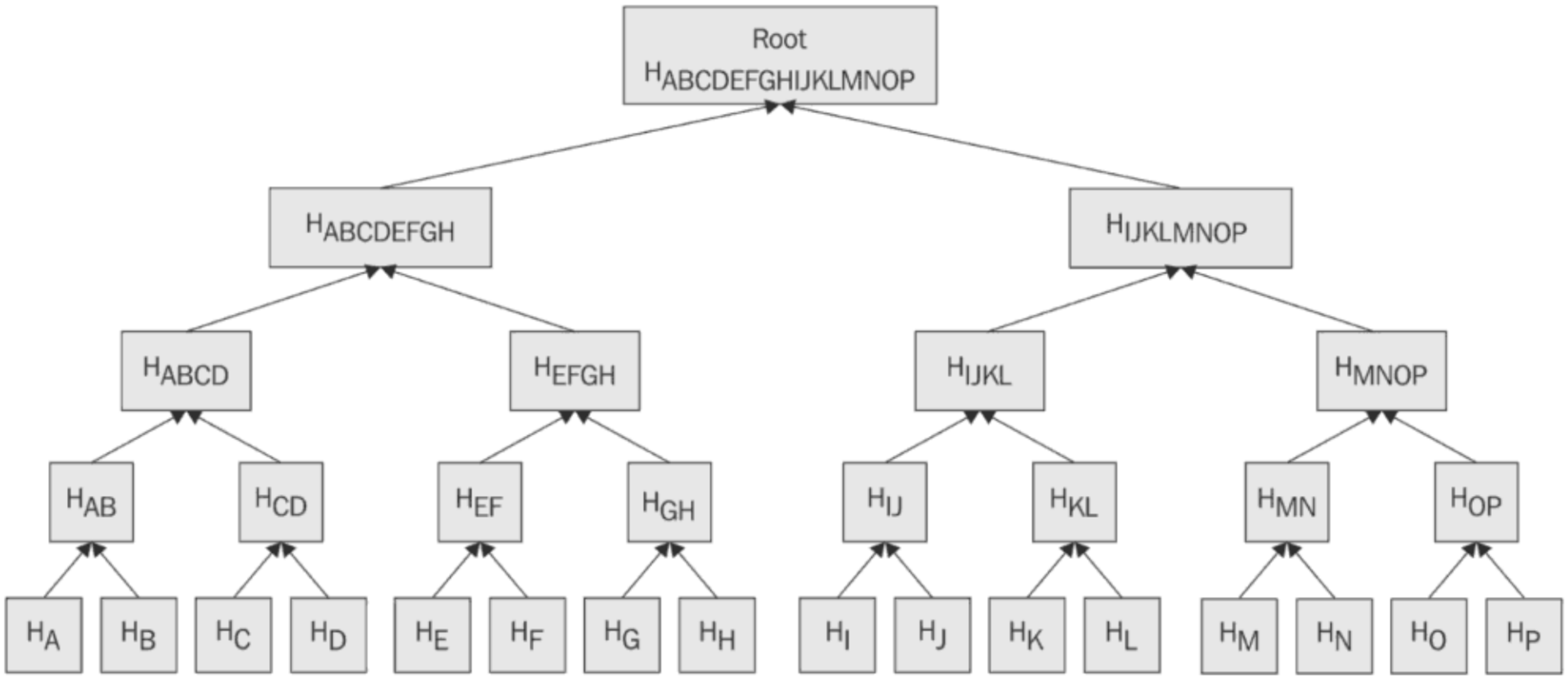


Encoding schemes

- Base 64
  - Used to encode binary data into printable characters
- Base 58
  - First introduced with Bitcoin
  - Used to encode integers into alphanumeric strings

Applications of hash functions

- Merkle trees
- Patricia trees
- Distributed hash tables



Exercise

- Find what public key cryptography capabilities are available in OpenSSL.
- Devise an experiment to see how OpenSSL can be used to demonstrate public key cryptography.

Summary

In this presentation, we covered:

- Mathematics concepts
- Asymmetric key cryptography
- RSA
- Elliptic Curve Cryptography (ECC)
- Zero-knowledge proofs
- Different types of digital signatures
- Merkle trees, Patricia trees and distributed hash tables